ACCESO A DATOS - 2.º DAM

UP 1 – Ficheros JSON

1. ¿Qué es JSON y para qué sirve?

JSON (*JavaScript Object Notation*) es un formato de texto utilizado para almacenar e intercambiar datos entre aplicaciones.

Aunque su nombre procede de JavaScript, es independiente del lenguaje de programación. Podemos utilizarlo desde Java, Python, JavaScript y muchos otros.

Por ejemplo, una aplicación Java puede guardar información en un fichero JSON para recuperarla posteriormente o recibir datos JSON enviados por un servidor a través de Internet.

Imagina que queremos compartir los datos de una persona entre una aplicación Java y una web. En lugar de inventar un formato propio, podemos escribirlos así:

```
{
  "nombre": "Pepito",
  "apellidos": "Grillo",
  "edad": 25,
  "activo": true
}
```

> IDEA CLAVE
>
> El formato JSON es independiente de Java: cualquier aplicación que entienda JSON podrá leer estos datos.

2. Estructura y tipos de datos

JSON organiza la información mediante **objetos y arrays**.

- **Objeto:** se representa entre llaves { } y contiene propiedades formadas por **pares clave-valor**. Las claves se escriben entre comillas dobles y se separan de sus valores mediante dos puntos (:).
- **Array:** se representa entre corchetes [ ] y contiene una lista ordenada de valores separados por comas.

Los valores pueden ser cadenas de texto, números, booleanos, null, objetos u otros arrays.

Ejemplo de un objeto JSON

El siguiente ejemplo almacena información de una persona:

```
{
  "nombre": "Pepito",
  "apellidos": "Grillo",
  "edad": 25,
  "altura": 1.75,
  "activo": true,
  "telefono": null,

  "aficiones": [
    "videojuegos",
```

```
    "cine",
    "música"
  ],

  "direccion": {
    "ciudad": "Alicante",
    "codigoPostal": "03001"
  }
}
```

Observa que un mismo objeto puede contener distintos tipos de datos e incluso otros objetos o arrays en su interior.

Tipos de datos en JSON

Elemento	Ejemploe n JSON	Significado
Objeto	`"direccion": { ... }`	Conjunto de propiedades clave-valor, delimitado por llaves.
Array	`["videojuegos", "cine", "música"]`	Lista ordenada de valores, delimitada por corchetes.
Cadena	`"Pepito"`	Texto entre comillas dobles.
Número	`25 / 1.75`	Valor numérico sin comillas.
Booleano	`true`	Valor lógico: true o false.
Nulo	`null`	Representa un valor nulo.

Errores habituales

En JSON debemos respetar algunas reglas de sintaxis:

- Las claves y las cadenas se escriben entre comillas dobles, nunca simples.
- No se permiten comentarios dentro del JSON.
- No debe aparecer una coma después de la última propiedad de un objeto ni del último elemento de un array.

Por ejemplo, el siguiente JSON tiene dos errores:

```
{
  'nombre': "Pepito",
  "edad": 25,
}
```

La clave `nombre` utiliza comillas simples y hay una coma innecesaria después de 25.

3. Objetos, Arrays y anidamientos

`JSON` permite crear estructuras de datos jerárquicas, en las que los objetos pueden contener otros objetos o arrays, y los arrays pueden contener objetos u otros arrays. Esta organización en distintos niveles se conoce como anidamiento.

Ejemplo: catálogo de videojuegos

Supongamos que queremos almacenar la información de varios videojuegos de LudotecaIES:

```
{
  "nombre": "LudotecaIES",
  "curso": "2º DAM",
  "videojuegos": [
    {
      "titulo": "Minecraft",
      "plataforma": "PC",
      "disponible": true
    },
    {
      "titulo": "Zelda",
      "plataforma": "Switch",
      "disponible": false
    }
  ]
}
```

En este ejemplo encontramos:

Elemento	Tipo	Significado
`nombre` y `curso`	Cadenas	Almacenan información del catálogo.
videojuegos	Array	Contiene una lista de videojuegos.
Cada elemento de videojuegos	Objeto	Contiene las propiedades titulo, plataforma y disponible.
disponible	Booleano	Indica si el videojuego está disponible (true) o no (false).

¿Cómo se organiza la información?

El JSON comienza con un objeto principal que contiene las propiedades nombre, curso y videojuegos.

La propiedad videojuegos es un array con dos elementos. Cada elemento es, a su vez, un objeto JSON que almacena los datos de un videojuego.

Los elementos de un array ocupan posiciones que comienzan en 0:

- Posición 0: Minecraft.
- Posición 1: Zelda.

Por ejemplo, para localizar la plataforma de Zelda, debemos acceder al array `videojuegos`, seleccionar el objeto de la posición 1 y consultar su propiedad plataforma.

Esta estructura permite almacenar varios videojuegos en un mismo documento JSON, manteniendo agrupada la información de cada uno.

4. Empezamos con `org.json` en Java

Hasta ahora hemos visto cómo se organiza la información en formato JSON. Para trabajar con estos datos desde Java utilizaremos la librería `org.json`, que permite crear, leer y modificar estructuras JSON.

Java no incluye las clases `JSONObject` y `JSONArray` en su biblioteca estándar, por lo que debemos incorporar esta dependencia al proyecto.

Las dos clases principales que utilizaremos son:

Clase	Descripción
`JSONObject`	Representa un objeto JSON formado por pares clave-valor.
`JSONArray`	Representa un array JSON que contiene una lista ordenada de valores.

Añadir la librería al proyecto Maven

En IntelliJ, abre el archivo pom.xml del proyecto e incorpora la siguiente dependencia dentro del bloque `<dependencies>`:

```
<dependency>
    <groupId>org.json</groupId>
    <artifactId>json</artifactId>
    <version>20240303</version>
</dependency>
```

Después, recarga las dependencias de Maven para que IntelliJ descargue la librería y permita utilizar sus clases.

A partir de este momento podremos importar `JSONObject` y `JSONArray` en nuestros programas Java:

```
import org.json.JSONObject;
import org.json.JSONArray;
```

5. Crear y consultar un `JSONObject`

La clase `JSONObject` permite crear y manipular objetos JSON desde Java. Cada objeto contiene un conjunto de propiedades formadas por pares clave-valor.

Los datos se almacenan inicialmente **en memoria**. Crear un `JSONObject` no significa que se haya creado un fichero en el disco.

Crear un objeto JSON

El siguiente ejemplo crea un objeto con los datos de una persona:

```
import org.json.JSONObject;

JSONObject persona = new JSONObject();
persona.put("nombre", "Pepito");
persona.put("apellidos", "Grillo");
persona.put("edad", 25);
persona.put("activo", true);

System.out.println(persona.toString(2));
System.out.println(persona.getString("nombre"));
System.out.println(persona.getInt("edad"));
```

El constructor new `JSONObject()` crea un objeto JSON vacío. Después, el método `put()` permite añadir propiedades indicando su clave y su valor.

Al ejecutar el programa se mostrará una representación JSON similar a esta:

```
{
  "nombre": "Pepito",
  "apellidos": "Grillo",
  "edad": 25,
  "activo": true
}
```

El método `toString(2)` convierte el objeto JSON en una cadena de texto con una sangría de dos espacios para facilitar su lectura. El orden de las propiedades puede variar.

Acceder a las propiedades

Para recuperar la información utilizamos distintos métodos según el tipo de dato:

```
String nombre = persona.getString("nombre");
int edad = persona.getInt("edad");
boolean activo = persona.getBoolean("activo");

System.out.println(nombre);
System.out.println(edad);
System.out.println(activo);
```

Cada método recibe la clave de la propiedad que queremos consultar y devuelve su valor en el tipo correspondiente.

Principales métodos de JSONObject

Método	¿Para qué se utiliza?
`put(clave, valor)`	Añadir o actualizar una propiedad
`getString(clave)`	Leer un texto
`getInt(clave)`	Leer un entero
`getDouble(clave)`	`Obtiene un valor numérico decimal.`
`getBoolean(clave)`	Leer un booleano
`getJSONObject(clave)`	Acceder a un objeto anidado
`getJSONArray(clave)`	Acceder a un array
`toString(2)`	Representar el JSON como texto con sangría

Los métodos `getJSONObject()` y `getJSONArray()` se utilizan cuando una propiedad contiene otra estructura `JSON`.

¿Qué sucede si una propiedad no existe?

Los métodos `get...()` esperan encontrar la clave solicitada y un valor que puedan interpretar con el tipo indicado.

Por ejemplo:

```
int edad = persona.getInt("edad");
```

Esta instrucción funciona si edad contiene un valor numérico adecuado. Si la clave no existe o el valor no puede convertirse al tipo solicitado, se producirá una excepción de org.json.

> **PARA PENSAR**
>
> ¿Qué sucedería al ejecutar getInt("edad") si el valor de edad fuera "veinticinco"? ¿Y si no existiera esa clave?

6. Trabajar con un `JSONArray`

La clase `JSONArray` permite crear y manipular listas ordenadas de valores en formato JSON. Sus elementos pueden ser cadenas, números, booleanos, objetos JSON u otros arrays.

Al igual que ocurre con los arrays de Java, **las posiciones comienzan en 0**.

Crear y recorrer un JSONArray

El siguiente ejemplo crea un array con varios nombres y muestra su contenido:

```
import org.json.JSONArray;

JSONArray nombres = new JSONArray();
nombres.put("Pepito Grillo");
nombres.put("Pinocho");
nombres.put("Gepetto");

for (int i = 0; i < nombres.length(); i++) {
    System.out.println(nombres.getString(i));
}
```

El método put() añade un elemento al array, mientras que length() devuelve el número de elementos que contiene. Para recuperar un valor concreto utilizamos su posición, por ejemplo, getString(0).

Arrays de objetos JSON

Un JSONArray también puede contener objetos JSONObject. Esto permite almacenar varios elementos que comparten una misma estructura.

Retomamos el catálogo de videojuegos del apartado 3. Suponemos que tenemos el JSON almacenado en un JSONObject llamado catalogo.

```
JSONArray videojuegos = catalogo.getJSONArray("videojuegos");

JSONObject videojuego = videojuegos.getJSONObject(0);
String titulo = videojuego.getString("titulo");

System.out.println(titulo);
```

Primero obtenemos el array asociado a la clave videojuegos. Después accedemos a su primer elemento, que es un objeto JSON, y recuperamos su título.

Para mostrar todos los videojuegos, podemos recorrer el array:

```
for (int i = 0; i < videojuegos.length(); i++) {
    JSONObject juego = videojuegos.getJSONObject(i);

    System.out.println(juego.getString("titulo"));
    System.out.println(juego.getString("plataforma"));
}
```

En este caso utilizamos getJSONObject(i) porque cada posición del array contiene un objeto JSON, no una cadena de texto.

Principales métodos de JSONArray

Método	Descripción
`put(valor)`	Añade un elemento al array.
`length()`	Devuelve el número de elementos.
`getString(i)`	Obtiene la cadena almacenada en la posición indicada.
`getInt(i)`	Obtiene un valor entero.
`getJSONObject(i)`	Obtiene el objeto JSON almacenado en esa posición.
`getJSONArray(i)`	Obtiene un array JSON contenido en otro array.

La diferencia respecto a JSONObject es que en un objeto accedemos a sus propiedades mediante claves, mientras que en un array accedemos a sus elementos mediante posiciones.

PARA PENSAR

En el catálogo de LudotecaIES, ¿por qué utilizamos primero getJSONArray("videojuegos") y después getJSONObject(i)? ¿Qué ocurriría si intentáramos utilizar getString(i) directamente sobre uno de los videojuegos?

7. Guardar JSON en un fichero

Hasta ahora hemos creado y manipulado objetos JSON en memoria. Sin embargo, estos datos se pierden cuando finaliza el programa si no los guardamos en un fichero.

Para almacenar la información de forma permanente, podemos convertir un JSONObject en una cadena de texto mediante toString() y escribirla en disco utilizando Files.writeString().

RECORDATORIO: ESCRITURA DE FICHEROS DE TEXTO

El método Files.writeString() de Java permite escribir una cadena de texto en un fichero.

- Si el fichero no existe, lo crea.
- Si ya existe, sobrescribe su contenido por defecto.
- Podemos indicar la codificación de caracteres, por ejemplo, StandardCharsets.UTF_8.

```
Path ruta = Path.of("datos.txt");
Files.writeString(ruta, "Hola mundo", StandardCharsets.UTF_8);
```

Este método ya lo hemos utilizado para trabajar con ficheros de texto. Ahora lo aplicaremos a JSON: la diferencia es que escribiremos el texto generado por JSONObject.toString().

Ejemplo: guardar un objeto JSON

El siguiente programa crea un objeto JSON con los datos de una persona y los guarda en un fichero llamado persona.json.

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.charset.StandardCharsets;
import java.io.IOException;
import org.json.JSONObject;

public class GuardarPersona {
    public static void main(String[] args) {
        JSONObject persona = new JSONObject();
        persona.put("nombre", "Pepito");
        persona.put("apellidos", "Grillo");

        Path ruta = Path.of("persona.json");
        try {
            Files.writeString(ruta, persona.toString(2),
                              StandardCharsets.UTF_8);
            System.out.println("Fichero guardado");
        } catch (IOException e) {
            System.out.println("Error al guardar: " + e.getMessage());
        }
    }
}
```

¿Cómo funciona?

Instrucción	Explicación
`new JSONObject()`	Crea un objeto JSON vacío en memoria.
`put(clave, valor)`	Añade las propiedades y sus valores.
`Path.of("persona.json")`	Representa la ruta donde se guardará el fichero.
`toString(2)`	Convierte el objeto JSON en texto con una sangría de dos espacios.

`Files.writeString()`	`Escribe el contenido en el fichero.`
`StandardCharsets.UTF_8`	`Establece la codificación de caracteres utilizada.`

¿Cuándo se crea el fichero?

La instrucción Path.of("persona.json") únicamente representa una ruta. El fichero se crea cuando se ejecuta Files.writeString(), siempre que la operación pueda realizarse correctamente.

Si el fichero no existe, se crea. Si ya existe, su contenido se sobrescribe por defecto.

En este ejemplo, persona.json se guardará en el directorio de trabajo desde el que se ejecuta el programa.

> **PARA PENSAR**
>
> ¿Qué ocurriría si ejecutamos el programa dos veces, cambiando el nombre de la persona en la segunda ejecución? ¿Se añadirían los nuevos datos al fichero o se sustituiría su contenido?

8. Leer e interpretar el fichero

En el apartado anterior hemos guardado los datos de una persona en el fichero persona.json. Ahora vamos a recuperar esa información desde Java.

Para ello, realizaremos dos operaciones:

1. **Leer el fichero:** Files.readString() recupera su contenido como una cadena de texto.
2. **Interpretar el JSON:** new JSONObject(texto) convierte esa cadena en un objeto JSON para poder acceder a sus propiedades.

Ejemplo: leer un fichero JSON

El siguiente programa recupera la información almacenada en persona.json y muestra sus datos por pantalla.

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.charset.StandardCharsets;
import java.io.IOException;
import org.json.JSONObject;

public class LeerPersona {
  public static void main(String[] args) {
    try {
      String texto=Files.readString(Path.of("persona.json"),StandardCharsets.UTF_8);
      JSONObject persona = new JSONObject(texto);
      System.out.println(persona.getString("nombre") + " "
                         + persona.getString("apellidos"));
        } catch (IOException e) {
            System.out.println("No se pudo leer el fichero");
        }
    }
}
```

¿Cómo funciona?

Instrucción	Explicación
`Path.of("persona.json")`	Representa la ruta del fichero que queremos leer.
`Files.readString()`	Lee el contenido completo del fichero como texto.
`StandardCharsets.UTF_8`	Indica la codificación utilizada para interpretar los caracteres.
`new JSONObject(texto)`	Interpreta el texto leído y crea un objeto JSON en memoria.
`getString() / getInt()`	Recuperan las propiedades indicando su clave.

Lectura del fichero e interpretación del JSON

Es importante distinguir ambas operaciones. **Leer correctamente un fichero no garantiza que su contenido sea un JSON válido.**

Por ejemplo, podemos encontrar dos situaciones diferentes:

Situación	Qué ocurre
El fichero persona.json no existe.	Se produce una excepción de lectura, como NoSuchFileException, que pertenece a IOException.
El fichero existe, pero su contenido no tiene una estructura JSON válida.	org.json produce una JSONException al intentar interpretarlo.

> PARA PENSAR
>
> ¿Qué ocurriría si modificamos manualmente el fichero persona.json y eliminamos las comillas de la propiedad nombre? ¿Se produciría el error al leer el fichero o al intentar interpretar su contenido?

9. Aplicación Ludoteca

Hasta ahora hemos trabajado con ficheros JSON que contienen los datos de una sola persona. Sin embargo, también podemos almacenar varios objetos dentro de un mismo fichero utilizando un JSONArray.

Vamos a recuperar la información de un catálogo de videojuegos almacenado en catalogo.json.

Estructura del fichero

El fichero contiene un objeto principal con el nombre del catálogo y un array de videojuegos:

```
{
  "nombre": "LudotecaIES",
  "videojuegos": [
    {
      "id": 1,
      "titulo": "Minecraft",
      "plataforma": "PC"
    },
    {
```

```
      "id": 2,
      "titulo": "Zelda",
      "plataforma": "Switch"
    }
  ]
}
```

Leer y recorrer el catálogo

El siguiente programa lee el fichero y muestra los videojuegos almacenados:

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.io.IOException;
import org.json.JSONObject;
import org.json.JSONArray;
import org.json.JSONException;

public class LeerCatalogo {

    public static void main(String[] args) {

        try {
            String texto = Files.readString(Path.of("catalogo.json"));
            JSONObject catalogo = new JSONObject(texto);
            JSONArray videojuegos = catalogo.getJSONArray("videojuegos");

            System.out.println(catalogo.getString("nombre"));

            for (int i = 0; i < videojuegos.length(); i++) {
                JSONObject juego = videojuegos.getJSONObject(i);

                System.out.println(juego.getInt("id") + " - "
                        + juego.getString("titulo") + " ("
                        + juego.getString("plataforma") + ")");
            }

        } catch (IOException e) {
            System.out.println("Error al leer el fichero: " + e.getMessage());
        } catch (JSONException e) {
            System.out.println("Error en el JSON: " + e.getMessage());
        }
    }
}
```

Resultado de la ejecución

```
LudotecaIES
1 - Minecraft (PC)
2 - Zelda (Switch)
```

¿Cómo funciona?

Instrucción	Explicación
`Files.readString()`	Recupera el contenido del fichero como texto.
`new JSONObject(texto)`	Interpreta el contenido JSON.
`getJSONArray("videojuegos")`	Obtiene el array que contiene los videojuegos.

`videojuegos.length()`	Devuelve el número de elementos del array.
`getJSONObject(i)`	Obtiene el videojuego situado en la posición indicada.
`getInt() / getString()`	Recuperan las propiedades de cada videojuego.

> **PARA PENSAR**
>
> ¿Por qué utilizamos getJSONArray("videojuegos") para acceder a la colección y después getJSONObject(i) para recuperar cada videojuego?
>
> ¿Qué tendríamos que modificar si quisiéramos mostrar también si cada videojuego está disponible?

10. Conversión entre JSON y objetos Java

Hasta ahora hemos trabajado directamente con JSONObject y JSONArray. Sin embargo, en una aplicación Java también podemos utilizar la información de un JSON para crear objetos de nuestras propias clases.

Un JSONObject no es una instancia de la clase Videojuego. Para obtener un objeto de esta clase, debemos recuperar los valores del JSON y utilizarlos para inicializar sus atributos.

10.1. De JSON a objeto Java

Partimos de la siguiente información:

```
{
  "titulo": "Minecraft",
  "plataforma": "PC",
  "anio": 2011
}
```

Supongamos que ya tenemos definida la clase Videojuego con estos tres atributos, su constructor y los métodos de acceso correspondientes.

Si la variable datos contiene el JSONObject anterior, podemos crear un objeto Java utilizando su constructor:

```
Videojuego juego = new Videojuego(
        datos.getString("titulo"),
        datos.getString("plataforma"),
        datos.getInt("anio")
);
```

En este caso, los métodos getString() y getInt() recuperan los valores del JSON, que se pasan como argumentos al constructor de Videojuego.

A partir de ese momento, podemos trabajar con juego como con cualquier otro objeto Java.

10.2. De objeto Java a JSON

También podemos realizar la operación inversa: obtener los atributos de un objeto Java y utilizarlos para crear un JSONObject.

```
JSONObject json = new JSONObject();
```

```
json.put("titulo", juego.getTitulo());
json.put("plataforma", juego.getPlataforma());
json.put("anio", juego.getAnio());

System.out.println(json.toString(2));
```

El resultado será un JSON con las propiedades correspondientes a los atributos del videojuego.

Diferencias entre ambas operaciones

Operación	Qué hacemos
JSON → objeto Java	Recuperamos las propiedades del JSONObject y creamos una instancia de nuestra clase.
Objeto Java → JSON	Obtenemos los atributos del objeto Java y los añadimos a un JSONObject.

PARA PENSAR

Si añadimos un atributo boolean disponible a la clase Videojuego, ¿qué cambios tendríamos que realizar para recuperar ese dato desde el JSON y volver a incluirlo al generar un nuevo JSONObject?